A Project Report On

Online Notes App

Submitted in partial fulfillment of the requirement for the BCA Semester-4 Python Programming Subject Final Examination.

BACHELOR OF COMPUTER APPLICATIONS
(B.C.A.)

Academic Year 2025 – 26

Aayush Kishorbhai Gadhiya - 92400527237

Internal Guide
Kishita Doshi

Marwadi University
Marwadi Chandarana Group
NAAC A+

Rajkot-Morbi Road, At & PO : Gauridad, Rajkot 360 003. Gujarat. India.

CONTENTS

Chapters	Particulars	Page No.
1	**SYNOPSIS**	
2 2.1 2.2 2.3 2.4	**PREAMBLE** General Introduction Statement of Problem Objective and Scope of the Study Module Description with functionality	
3 3.1 3.2 3.3 3.4	**SYSTEM DESIGN AND DEVELOPMENT** Data Flow Diagram (DFD) Table Structure Menu Design Screen Design (Screen shots with short description)	
4	**CONCLUSION**	
5	**LEARNING DURING PROJECT WORK**	
6 6.1 6.2	**BIBLIOGRAPHY** Online References Offline References	

FIGURE INDEX

Sr. No.	Figure No.	Particulars	Page No.
1	Figure 1	Login Screen	
2	Figure 2	Registration Screen	
3	Figure 3	Dashbord (Light Mode / Dark Mode)	
4	Figure 4	Add New Note	
5	Figure 5	Edit Note	
6	Figure 6	Trash Page	
7	Figure 7	Share Note Popup	
8	Figure 8	Public Shared Note View	
9	Figure 9	Profile Page	

TABLE INDEX

Sr. No.	Table No.	Particulars	Page No.
1	Table 1	User Table Structure	
2	Table 2	Note Table Structure	
3	Table 3	Tag Table Structure	
4	Table 4	Note_Tag Association Table	

Chapter 1: Synopsis

NovaNotes is a feature-rich, web-based note-taking application designed for professionals who need to organise, prioritise, and retrieve their notes efficiently. Built with Flask (Python) and SQLite, it offers:

- **User authentication (register/login)**
- **Rich notes with Markdown support**
- **Reminders, expiry dates, due dates**
- **Priority levels (High/Medium/Low)**
- **Tags and full-text search**
- **Archive, soft delete (trash), and permanent delete**
- **Share notes via public links**
- **Export notes as JSON**
- **Dark/light theme with persistence**
- **Responsive, glassmorphic UI**

The system solves the problem of scattered, unstructured note-keeping by providing a centralised, secure, and intelligent workspace. It is easy to deploy, scalable, and suitable for individual users.

Chapter 2: Preamble

2.1 General Introduction

In the digital age, note-taking has evolved from paper to digital platforms. However, most free tools lack advanced features like reminders, expiry handling, priority tagging, and easy sharing. NovaNotes bridges this gap by offering a professional yet simple solution. Using modern web technologies (Flask backend, SQLite database, HTML/CSS/JS frontend with glassmorphism), the application is lightweight, fast, and runs on any device.

2.2 Statement of Problem

· Users forget important notes because no reminder system exists.
· Notes accumulate without expiry or archiving, causing clutter.
· Searching through hundreds of notes is tedious without full-text search and tagging.
· Sharing notes often requires copying/pasting content; no secure, link-based sharing.
· Most free note apps lack priority levels and due dates for task-oriented notes.
· Users want a visually appealing, dark-mode supported interface.

2.3 Objective and Scope of the Study

Objectives:

· Develop a secure, multi-user note management system.
· Implement reminder, expiry, and due date features.
· Provide tagging, priority levels, and full-text search.
· Enable note sharing via unique tokens.
· Build a responsive, modern UI with light/dark theme.

Scope:

· The system is web-based and accessible from any browser.
· It supports only text notes (no image/file attachments initially).
· Email reminders are optional (can be configured via SMTP).
· The application is designed for individual users; team collaboration is out of scope.

2.4 Module Description with Functionality

Module Functionality
Authentication User registration, login, logout, password hashing, session management using Flask-Login.
Dashboard Displays notes with filters (all, expired, priority, archived), search bar, and quick actions (edit, delete, archive, share).

Note Management Create, edit, soft delete, restore, permanent delete, archive/unarchive notes.
Advanced Fields Set reminder time, expiry date, due date, priority (high/medium/low), and tags (comma separated).
Search & Filters Client-side and server-side search by title/content; filter by priority, expiry, archived.
Sharing Generate a random token per note, create a read-only public link.
Trash View deleted notes, restore them, or permanently delete them.
Profile Update email, timezone, password, and export all notes as JSON.
Theme Toggle Switch between light and dark mode; preference saved in localStorage.
Reminders (optional) Background scheduler checks for reminder times and sends email alerts (if configured).

Chapter 3: System Design and Development

3.1 Data Flow Diagram (DFD)

Context Level (Level 0):
The user interacts with the system to register, login, create/edit/delete notes, share notes, and export data. The system stores data in a database and sends email reminders (optional).

```
[User]
  ↓
(Login/Register Request)
  ↓
[Flask App (app.py)]
  ↓
[Database (models.py / SQLite)]
  ↑
(Response: Notes / Profile / Auth Status)
  ↑
[User]
```

[User]
↓
(HTTP Request: Login / Register / Add Note / Edit Note)
↓
[app.py Routes]
↓
[Authentication Logic]
↓
[Database Access (models.py)]
↓
[Templates (HTML Pages)]
↓
(Response to User)

[app.py]
↓
(Request: Save / Fetch Data)
↓
[models.py]
↓
[Database Tables]
↓
(User Table / Notes Table)
↑
(Return Data)
↑
[app.py]

[User]
↓
(Fill Note Form)
↓
[add_note.html]
↓
[app.py → add_note()]
↓
[models.py → Insert Note]
↓
[Database]

↓
(Success Response)
↓
[Dashboard Page]

[User]
↓
(Edit Request)
↓
[app.py → edit_note()]
↓
[Fetch Old Note from DB]
↓
[Update Note]
↓
[Database]
↓
[Updated Note Display]

[User]
↓
(Login/Register Form)
↓
[app.py]
↓
[models.py]
↓
[Check / Insert User in DB]
↓
[Auth Result]
↓
[Dashboard OR Error Message]

[User]
↓
(Delete Note Click)
↓
[app.py]
↓

[models.py]
↓
[Move to Trash / Delete]
↓
[Database]
↓
[Trash Page Update]

[User A]
↓
(Share Note)
↓
[app.py]
↓
[models.py]
↓
[Database (Shared Entry)]
↓
[User B Access]
↓
[View Shared Note Page]

3.2 Table Structure (Database Schema)

The system uses SQLite with the following tables:

Table 1: user

Column Type Description

id INTEGER PRIMARY KEY Auto-increment

username VARCHAR(80) UNIQUE Login name
email VARCHAR(120) UNIQUE For reminders
password_hash VARCHAR(200) Hashed password
timezone VARCHAR(50) e.g., 'Asia/Kolkata'
created_at DATETIME Registration time

Table 2: note

Column Type Description

id INTEGER PRIMARY KEY Auto-increment
title VARCHAR(200) Note title
content TEXT Note body (Markdown)
created_at DATETIME Creation timestamp
updated_at DATETIME Last edit timestamp
reminder_time DATETIME When to remind
expiry_date DATETIME After this, note is expired
due_date DATETIME Optional deadline
priority VARCHAR(20) 'high','medium','low'
is_archived BOOLEAN Archived flag
is_deleted BOOLEAN Soft delete flag
deleted_at DATETIME When moved to trash
share_token VARCHAR(64) Unique random token
user_id INTEGER Foreign key to user.id

Table 3: tag

Column Type Description
id INTEGER PRIMARY KEY Auto-increment
name VARCHAR(50) UNIQUE Tag name

Table 4: note_tags (association table)

Column Type
note_id INTEGER (FK to note.id)
tag_id INTEGER (FK to tag.id)

(Refer Table Index for page numbers)

3.3 Menu Design

The application uses a top navigation bar (glassmorphic) with the following options:

- **Logo – NovaNotes (links to Dashboard)**
- **+ New Note – opens Add Note page**
- **□ Theme Toggle – switches dark/light mode**
- **□ Trash – view deleted notes**
- **□ Profile – user settings and export**
- **□ Logout – ends session**

On the dashboard, there are filter chips: All, Expired, High Priority, Archived.

Each note card has action icons: ✏ Edit, □ Delete (soft), □ Archive/Unarchive, □ Share.

3.4 Screen Design (Screenshots with short description)

8:17 96%

Create account

Start organising your ideas professionally

Username

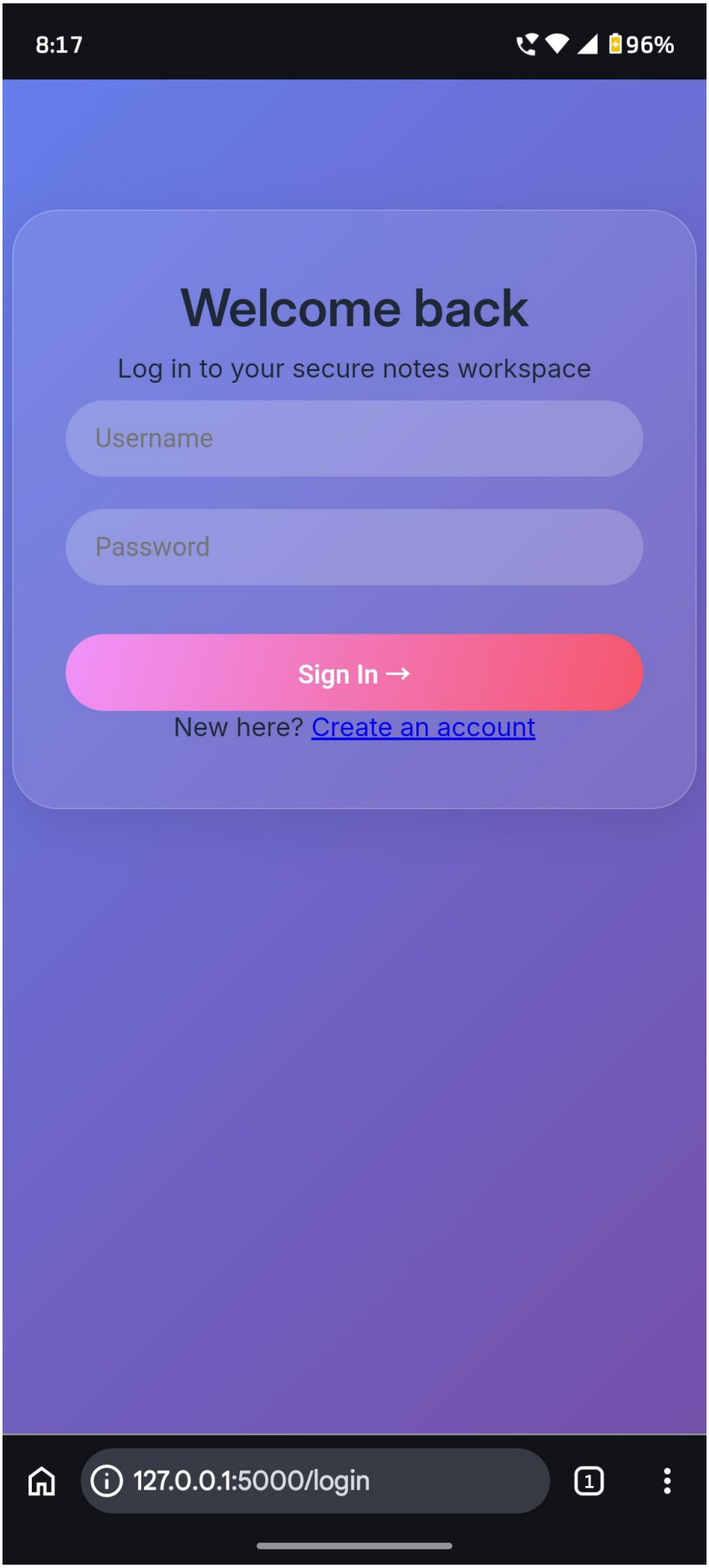
Email

Password

Get Started →

Already have an account? Sign in

127.0.0.1:5000/register


8:17
96%
Welcome back
Log in to your secure notes workspace
Username
Password
Sign In →
New here? Create an account
127.0.0.1:5000/login

8:17 96%

NovaNotes

+ New Note

Search notes...

All Expired High Archived

No notes yet. Create your first note

127.0.0.1:5000/dashboard

8:17 96%

🗑️ Trash

← Back to Notes

🗑️

Trash is empty. Deleted notes appear here.

Go to Notes

127.0.0.1:5000/trash

8:17 96%

👤 Your profile

← Dashboard

Username

aayushgadhiya@07

Username cannot be changed

Email address

aayush.gadhiya128830@marwadiuniversity.ac.in

Timezone (e.g., Asia/Kolkata, America/New_York)

UTC

Used for correct reminder times

New password (leave blank to keep current)

Enter new password

Save Changes

📥 Export all notes (JSON)

127.0.0.1:5000/profile

Chapter 4: Conclusion

NovaNotes successfully delivers a professional note-taking platform with all planned features. The use of Flask ensures rapid development, while SQLite provides a lightweight database. The glassmorphic UI with dark mode enhances user experience. Advanced features like reminders, expiry dates, priority levels, tags, and shareable links make it superior to basic note apps. The system is easy to install and run, requiring only Python and a few libraries. Future enhancements could include file attachments, real-time collaboration, and mobile apps.

Chapter 5: Learning During Project Work

· Full-stack web development – integrating Flask backend with HTML/CSS/JS frontend.
· Database design – normalisation, relationships (many-to-many via association table).
· User authentication – password hashing (werkzeug), session management (Flask-Login).
· Soft delete pattern – using is_deleted flag instead of permanent deletion.
· Markdown rendering – converting markdown to HTML securely.
· Background scheduling – using APScheduler for reminder checks.
· Responsive UI – CSS Grid, Flexbox, media queries.
· Dark mode implementation – CSS variables and localStorage.
· Error handling – flash messages and graceful fallbacks.
· Version control – using Git for source code management.

Chapter 6: Bibliography

6.1 Online References

1. Flask Documentation – https://flask.palletsprojects.com/

2. Flask-SQLAlchemy – https://flask-sqlalchemy.palletsprojects.com/

3. Flask-Login – https://flask-login.readthedocs.io/

4. Markdown Python library – https://python-markdown.github.io/

5. MDN Web Docs – CSS Grid, Flexbox, localStorage – https://developer.mozilla.org/

6. APScheduler – https://apscheduler.readthedocs.io/

7. SQLite Official Site – https://www.sqlite.org/

6.2 Offline References

1. Flask Web Development, 2nd Edition – Miguel Grinberg (O'Reilly)

2. Python Crash Course, 2nd Edition – Eric Matthes (No Starch Press)

3. HTML & CSS: Design and Build Websites – Jon Duckett (Wiley)